PROJET 01 — ORGANISATION

Répartition des tâches

Projet Rootkit — Noyau Linux · groupe de 4

Module : Noyau Linux · École 2600 · 2026
Document interne d'organisation — 4 segments, planning & dépendances
Rendu attendu : image QCOW2 (LFS) + sources + documentation + rapport de conception

Sommaire

1. But de ce document & méthode de découpage
2. Vue d'ensemble : les 4 segments
3. Segment A — Infrastructure LFS, image & persistance
4. Segment B — Cœur noyau : module & canal de commande
5. Segment C — Furtivité & camouflage
6. Segment D — Compagnon userland, fonctionnalités & documentation
7. Dépendances entre segments
8. Planning indicatif (5 semaines — jusqu'au 1er novembre)
9. Tableau de ratio (à remplir ensemble)
10. Couverture des livrables imposés
11. Responsabilités transverses & rituels d'équipe

1. But de ce document & méthode de découpage

Ce document sert à **répartir équitablement** le travail du projet Rootkit entre les **4 membres du groupe**. Il ne contient **aucun code** ni technique d'implémentation : c'est un outil d'organisation. Il propose 4 segments, leurs dépendances, un planning et un tableau de ratio ; **l'attribution nominative se décidera ensemble** — les cases « Titulaire » sont volontairement vides.

Principe de découpage. On coupe le projet selon les **frontières techniques naturelles** (ce qui peut avancer en parallèle avec un minimum de couplage), pas selon les fonctionnalités du sujet une à une. Chaque segment est une **surface de responsabilité** cohérente, testable seule, avec un livrable propre. Objectif : que chacun puisse travailler sans attendre en permanence les autres, et qu'aucun segment ne concentre à lui seul toute la difficulté.

Contraintes du sujet prises en compte. Le rapport de conception doit inclure la répartition et un **tableau de ratio en %** (base 100 % = travail normal), utilisé pour la notation. Ce document prépare exactement ce livrable. Rappel des interdits structurants qui orientent le découpage : ne pas détourner l'appel `kill` ; rendre un travail **original** au groupe (tout code similaire entre groupes = 0) ; identifiants en anglais ; style de code conforme aux recommandations ANSSI (C). Tout se déroule dans la **VM QEMU jetable** fournie par la LFS.

Critères d'équilibrage

Critère	Ce qu'on vise
Charge	~25 % du volume total par personne (le ratio final ajuste au réel).
Difficulté	Chaque segment mélange une part « facile » et une part « exigeante ».
Autonomie	Un segment peut démarrer sans attendre que les 3 autres soient finis.
Intégration	Interfaces entre segments définies tôt (voir §7) pour limiter le couplage.
Rédaction	Chacun rédige la partie du rapport correspondant à son segment.

2. Vue d'ensemble : les 4 segments

Le projet se lit comme **quatre couches** qui s'emboîtent : le **terrain** (le système de test et son démarrage), le **moteur** (le module et sa façon de recevoir des ordres), la **discrétion** (rester invisible) et la **vitrine** (les outils côté utilisateur qui démontrent le tout, plus la doc). Chaque membre « possède » une couche mais contribue à l'intégration commune.

A

Infra LFS, image & persistance

Le terrain de démonstration : build reproductible + démarrage.

Scripts de fabrication de la LFS, deux comptes (root/user), conversion en QCOW2 compressé, mécanisme de chargement automatique du module au démarrage, build-system global.

B

Cœur noyau : module & canal

Le moteur : squelette du LKM et réception des commandes.

Ossature du module `.ko`, cycle chargement/déchargement propre, mécanisme de **remplacement** d'appels système existants, canal compagnon→module, répartition des commandes.

C

Furtivité & camouflage

La discrétion : rester invisible pour l'espace utilisateur.

Dissimulation du module, des fichiers, des processus et des artefacts liés à la persistance, de sorte qu'un utilisateur « non initié » ne voie rien d'anormal.

D

Compagnon, fonctionnalités & doc

La vitrine : les programmes userland + la documentation.

Programme(s) compagnon, jeu de fonctionnalités déclenchées via le canal, scénarios de démonstration end-to-end, documentation utilisateur (comptes, usage).

Lecture des couleurs. Chaque segment garde sa couleur dans tout le document (planning, dépendances, ratio) : A violet, B bleu, C orange, D vert.

3. Segment A — Infrastructure LFS, image & persistance

Mission. Fournir le **terrain** : un système Linux de test que l'on reconstruit par script, qui démarre dans QEMU et sur lequel le rootkit s'installe et se relance tout seul. C'est le socle sur lequel les trois autres démontrent leur travail.

Périmètre

- Scripts de fabrication de la **LFS** (reproductibles, versionnés), à partir du TP01.
- Deux comptes conformes au sujet : **root** (chargement du `.ko`) et **user** (démonstration via le compagnon) ; mots de passe documentés côté Segment D.
- Conversion de l'image « raw » en **QCOW2 compressé** (`qemu-img convert -c`) — c'est le format de rendu imposé, sous peine de pénalité.
- Mécanisme de **persistance** : le module se charge automatiquement à chaque démarrage (script d'init) — en coordination étroite avec le Segment C qui devra masquer cette trace.
- **Build-system global** : une commande qui reconstruit image + modules + compagnons, et produit le livrable final prêt à rendre.

Livrables du segment

- Dossier de scripts LFS + procédure de reconstruction.
- Image QCOW2 compressée finale (assemblage des sources des autres segments).
- Script/Make d'orchestration de bout en bout.
- Chapitre « fabrication & démarrage » du rapport.

> **Point d'attention.** La persistance et la furtivité sont **couplées** : la trace laissée au démarrage (A) doit être masquable (C). Décider tôt **où** vit le déclencheur de chargement pour que C sache quoi cacher.

4. Segment B — Cœur noyau : module & canal de commande

Mission. Construire le **moteur** : l'ossature du module noyau et la façon dont il **reçoit des ordres** du programme compagnon. C'est la brique centrale ; A l'installe, C le rend invisible, D lui parle.

Périmètre

- Squelette du **LKM** : `init` / `exit`, cycle de vie propre, robustesse (« aucun bogue susceptible de planter le système » est un objectif du sujet).
- Technique de **remplacement** d'appels système existants (le sujet impose de remplacer plutôt que d'ajouter ; **l'appel `kill` est interdit**).
- Conception du **canal** compagnon → module (par ex. via un appel système remplacé, un pseudo-fichier `dev`, ou autre) et du **protocole de commandes**.
- **Dispatcher** : associe une requête reçue à une action ; expose une interface interne stable que le Segment D consomme et que le Segment C réutilise pour ses hooks.

Livrables du segment

- Source du module + build-system (Makefile).
- Spécification du protocole de commandes (contrat d'interface avec C et D).
- Chapitre « architecture du module & canal » du rapport.

> **Rôle pivot.** B définit les **interfaces** que C et D consomment. Sa première priorité n'est pas la richesse fonctionnelle mais un **contrat clair** publié tôt (nom des commandes, format des arguments, valeurs de retour), pour débloquer les autres.

5. Segment C — Furtivité & camouflage

Mission. Rendre l'ensemble **discret** : un utilisateur de l'espace utilisateur qui ne connaît pas les méthodes d'accès ne doit rien remarquer. C'est le segment le plus « recherche » : beaucoup d'observation, d'essais et de vérifications.

Périmètre

- Dissimulation du **module** lui-même vis-à-vis des outils userland usuels.
- Dissimulation de **fichiers/répertoires** appartenant au rootkit (dont ceux du Segment A).
- Masquage de la **ligne de persistance** ajoutée par A dans les fichiers de démarrage.
- Masquage des **processus** et artefacts liés au compagnon (coordination avec D).
- Batterie de **tests d'invisibilité** : liste des vérifications « avant/après » à passer.

Livrables du segment

- Modules/hook de camouflage intégrés au cœur de B.
- Grille de tests de furtivité (ce qui doit rester invisible et comment on le vérifie).
- Chapitre « stratégie de discrétion » du rapport.

> **Dépendance forte.** C s'appuie sur les interfaces de B et sur les traces concrètes de A. Il démarre par l'**étude et la conception** (ce qu'il faudra cacher) pendant que B/A posent leurs bases, puis intègre. Prévoir des points de synchro fréquents.

6. Segment D — Compagnon userland, fonctionnalités & documentation

Mission. La **vitrine** : les programmes côté utilisateur qui pilotent le rootkit via le canal de B, le **jeu de fonctionnalités** à démontrer, les scénarios de démo bout-en-bout et la **documentation utilisateur**. C'est ce que le correcteur manipulera pour évaluer.

Périmètre

- Programme(s) **compagnon** (langage libre, style idiomatique du langage choisi).
- Implémentation, côté client, des **fonctionnalités** exposées par le module (via le protocole de B) — le catalogue précis se décide en équipe.
- **Scénarios de démonstration** reproductibles : de la connexion du user jusqu'à l'effet obtenu.
- **Documentation utilisateur** : comptes (login/mdp root & user), procédure de démonstration, mode d'emploi du compagnon.

Livrables du segment

- Sources du/des compagnon(s) + build-system.
- Documentation utilisateur (livrable imposé).
- Jeu de scénarios de démo + chapitre « usage & fonctionnalités » du rapport.

> **Bonus rapport.** Le sujet propose un bonus : présenter des scénarios crédibles d'accès initial et les domaines industriels où un noyau/modules non signés sont réalistes. C'est une partie **purement rédactionnelle** — bon candidat pour équilibrer la charge de qui aura le segment le plus léger au moment du bilan (voir §11).

7. Dépendances entre segments

Le couplage principal passe par le **contrat d'interface de B** (protocole de commandes) et par les **traces de A** que C doit masquer. Tout le reste peut avancer en parallèle.

```
A (LFS + persistance) ──installe──▶ B (module .ko) B (canal + protocole)
──interface──▶ C (hooks de camouflage) B (canal + protocole) ──interface──▶ D
(compagnon) A (traces au boot) ─────à cacher──▶ C (masquage) D (artefacts user)
─────à cacher──▶ C (masquage) tous ──────────────────assemble─────▶ A (image QCOW2
finale + build global)
```

Segment	A besoin de…	Fournit à…	Peut démarrer seul ?
A	—	Terrain à tous ; assemble le rendu final	Oui, immédiatement
B	Terrain de A (tôt, pas bloquant)	Contrat d'interface à C et D	Oui (dev sur VM standard)
C	Interfaces de B, traces de A	Discrétion à l'ensemble	Étude oui, intégration après B
D	Contrat d'interface de B	Démo & doc au correcteur	Maquette oui, brancher après B

Jalon débloquant n°1. Publier le **contrat d'interface de B** (liste des commandes + format) en fin de semaine 2. C'est le jalon qui libère C et D. À traiter en priorité absolue, même sous forme minimale.

8. Planning indicatif (5 semaines — 27 sept. → 1er nov.)

Barres = période active du segment ; ■ = jalon commun. Découpage réel : du **27 septembre** au **1er novembre 2026** (5 semaines de 7 jours), le rendu tombant en fin de S5.

Semaine	S1 27/9–3/10	S2 4–10/10	S3 11–17/10	S4 18–24/10	S5 25/10–1/11
A LFS / image / persistance					
B Module & canal					
C Furtivité					
D Compagnon & doc					
Jalons communs		J1	J2	J3	J4

Jalon	Semaine	Contenu
J1	S2 — 10 oct.	Contrat d'interface de B publié ; LFS bootable minimale ; maquette compagnon.
J2	S3 — 17 oct.	Chaîne complète « charge → commande → effet » qui marche sur la VM (bout en bout).
J3	S4 — 24 oct.	Furtivité intégrée & persistance masquée ; gel des fonctionnalités.
J4	S5 — 1er nov.	QCOW2 compressé final, doc utilisateur, rapport + tableau de ratio, relecture ANSSI. **Rendu.**

9. Tableau de ratio (à remplir ensemble)

Livrable imposé pour la notation. Base : **100 % = travail normal d'un étudiant normal** ; un « fantôme » à 0 % aura 0. À compléter **honnêtement et collectivement** en fin de projet, une fois le réel connu. Les noms et pourcentages ci-dessous sont des espaces à remplir.

Membre	Segment principal	Contributions secondaires	Ratio %
.....	A Infra / image / persistance	.....	**... %**
.....	B Module & canal	.....	**... %**
.....	C Furtivité	.....	**... %**
.....	D Compagnon & doc	.....	**... %**

Conseil. Tenir un petit journal de contributions au fil de l'eau (qui a fait quoi, commits, chapitres rédigés) : le tableau de ratio se remplit alors sans conflit, avec des faits. Les valeurs intermédiaires sont libres — l'important est qu'elles reflètent le réel.

10. Couverture des livrables imposés

Vérification que chaque exigence du sujet a un **propriétaire**. Aucune case ne doit rester orpheline.

Livrable / exigence du sujet	Segment porteur	Contributeurs
Scripts de fabrication de la LFS	A	—
Image QCOW2 **compressée** (format imposé)	A	tous (sources)
Comptes root + user	A	D (doc mdp)
Persistance au démarrage	A	C
Source du rootkit + build-system	B	C
Remplacement d'appels système (pas `kill`)	B	C
Canal de communication compagnon ↔ module	B	D
Camouflage (module, fichiers, processus)	C	B
Source du/des programme(s) compagnon(s) + build	D	—
Fonctionnalités démontrées	D	B
Documentation utilisateur	D	A
Rapport de conception (chapitres)	tous	chacun son segment
Tableau de ratio %	tous	collectif
Conformité style de code (ANSSI, anglais)	relecteur désigné	tous
Bonus : scénarios d'accès initial / domaines industriels	à attribuer (§11)	rédaction

11. Responsabilités transverses & rituels d'équipe

Quelques rôles **ne rentrent dans aucun segment** mais doivent avoir un titulaire. On les distribue pour **rééquilibrer** la charge : celui dont le segment est le plus léger prend le rôle transverse le plus lourd.

Rôle transverse	Ce que ça couvre	Suggestion d'attribution
Intégrateur / release	Assemble l'image finale, gère le build global, produit le rendu.	Titulaire de A
Gardien des interfaces	Maintient le contrat de commandes à jour, arbitre les changements.	Titulaire de B
Relecteur qualité/ style	Passe ANSSI/style, identifiants en anglais, cohérence.	Titulaire de C ou D
Éditeur du rapport	Assemble les chapitres, homogénéise, tient le tableau de ratio.	Titulaire de D
Rédacteur du bonus	Scénarios d'accès initial + domaines industriels.	Charge la plus légère au bilan

Rituels proposés

- **Point hebdo** (15 min) : avancement, blocages, prochain jalon.
- **Journal de contributions** partagé : alimente le tableau de ratio sans dispute.
- **Dépôt commun** avec un dossier par segment + un dossier `rapport/`.
- **Revue croisée** avant chaque jalon : chacun relit le segment d'un autre (attrape les bugs et les écarts de style tôt).

> **À décider ensemble dès la première réunion.** (1) Qui prend A/B/C/D. (2) La date de rendu réelle → recaler le planning du §8. (3) Le langage du compagnon. (4) La forme du canal de commande (cadre le travail de B). Le reste découle de ces quatre choix.

Document d'organisation interne — sans code ni détail d'implémentation. À réviser après la première réunion de groupe et à joindre, sous sa forme finale (ratio rempli), au rapport de conception.